

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো
(Prof. Chester Rebeiro)
কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering)
ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ
(Indian Institute of Technology, Madras)
সপ্তাহ – 06
বক্তৃতা – 24
সমন্বয়সাধন
(Synchronization)

হ্যালো গত ভিডিওতে আমরা IPC বা Inter Process Communication দেখেছি। আমরা দেখেছি যে IPC এর কারণে আমরা বেশ কয়েকটি প্রসেস নিয়ে গঠিত application তৈরি করতে পারি এবং আমরা modularity অর্জন করতে পারি। এবং modular process থাকার ফলে, যেখানে প্রতিটি প্রক্রিয়া নির্দিষ্ট কাজ করে আমরা সহজে বোঝা যায় এবং খুব দক্ষ একটি application পেতে পারি। এই modular পদ্ধতির এবং IPC synchronization এর জন্য প্রয়োজন। এই ভিডিওতে আমরা synchronization কি দেখতে পাব এবং এর সাথে সংশ্লিষ্ট সমস্যাগুলি কী কী এবং এটি কিভাবে সমাধান করা যায় আলোচনা করব।

(স্লাইড সময় 01:03 দেখুন)

আসুন আমরা এই প্রেরণামূলক দৃশ্যপটে synchronization ব্যাখ্যা করি। ধরা যাক দুটি প্রোগ্রাম আছে; প্রোগ্রাম 0 এবং প্রোগ্রাম 1, এবং একটি shared variable আছে যা গণক (counter) হিসাবে সংজ্ঞায়িত করা হয়। এটি একটি global shared variable, `int counter = 5`। প্রোগ্রাম 0 তে counter টি 1 বৃদ্ধি করা হয়, এবং প্রোগ্রাম 1 এ counter টি 1 হ্রাস করা হয়। যখন আমরা এই প্রোগ্রাম single core processor এ চালাবো, প্রোগ্রাম 0 কিছু সময়ের জন্য চলবে, তারপর একটি context seitch হবে, তারপর প্রোগ্রাম 2 কিছু সময়ের জন্য চলবে এবং

তারপর প্রোগ্রাম 1 চলবে, এবং এইভাবে চলতে থাকবে। ধরে নেওয়া হচ্ছে যে এটা একটি round robin scheduling এবং অন্য কোন প্রসেস এখানে নেই। এখন প্রশ্ন হচ্ছে counter এর মান কত হবে?

(স্লাইড সময় 02:03 দেখুন)

যে কেউ আশা করবে যে counter এর মান 5 হবে। ধরা যাক প্রোগ্রাম 0 টি প্রথমে execute হবে, তাই এটি counter এর মান বাড়িয়ে 6 করবে। এবং তারপর প্রোগ্রাম 1 execute হয় এবং এটি counter এর মান হ্রাস করে পুনরায় 5 করবে যেহেতু counter এর মান 6 ছিল এটা কমে 5 হবে। অন্য দিকে, যদি প্রোগ্রাম 1 প্রথমে এবং তারপর প্রোগ্রাম 0 execute করে তাহলে counter--, counter এর মান 5 থেকে কমিয়ে 4 করবে এবং তারপর প্রোগ্রাম 0 আবার counter এর মান 4 থেকে বাড়িয়ে 5 করবে। সুতরাং, আশা করা যায় যে উভয় প্রোগ্রামের শেষে counter এর মান 5 হবে। কিন্তু এখন আমরা দেখব যে, আমরা counter এর মান 4 বা 6 ও পেতে পারি। তাই আসুন বিস্তারিতভাবে এটিকে দেখি।

(স্লাইড সময় 03:08 দেখুন)

সুতরাং, আসুন আরও বিস্তারিত ভাবে দেখি কিভাবে এই নির্দেশাবলী কার্যকর করা হচ্ছে। দেখা যাক যখন আমরা এই counter বৃদ্ধি করছি তখন কি ঘটে। যখন counter++ করছি তখন assembly instruction গুলি কি হয় দেখা যাক। প্রথমে, counter এর মান যেটি মেমরিতে সংরক্ষিত আছে সেটি একটি রেজিস্টারে, মনে করি R1 এ load হয়, এবং তারপর R1 1 দ্বারা বাড়ানো হয়। এটি দ্বিতীয় লাইন R1 সমান R1 প্লাস 1 এবং তারপর R1 এর মান মেমরিতে লেখা হবে। R1 মেমরির counter ভ্যালুর মধ্যে ফিরে আসে। সুতরাং সংখ্যা অনুযায়ী, আমাদের এখানে 5 টি হল counter তাই counter এর মান 5 এবং এটি R1 তে লোড হয়, তারপর 5 বাড়ানো হয় তাই এখন R1 তে 6 এবং R1 এর মান counter এ ফিরে আসে। সুতরাং, মেমরিতে 6 মানটি counter এর দ্বারা চিহ্নিত মেমরিতে ফিরে আসে। মনে করি এখন একটি context switch হল, আবার একই জিনিস হবে। সুতরাং counter এর মান register R2 এর মধ্যে লোড করা হয়, তাই R2 এর মান হয় 6 তারপর R2 কে 1 হ্রাস করা হয়, সুতরাং R2 হল 5। এবং তারপর রেজিস্টার R2 এর মান আবার মেমরি অবস্থান counter এর মধ্যে সংরক্ষিত হয়। সুতরাং এই দুটি প্রোগ্রামের শেষে counter এর মান হয় 5। এখন আমরা দ্বিতীয় দুইটি পরিস্থিতির দিকে তাকাব, যেখানে কাউন্টার মান 4 এবং 6 হিসাবে পেতে পারি। মনে করি প্রোগ্রাম 0 চালানো হল এবং R1 মধ্যে counter এর মান লোড হল; মনে রাখবেন যে R1 এর মান আছে 5। এবার একটি context switch হল এবং প্রোগ্রাম 1 চালানো হল। এখন এখানে counter এখনও 5 মান আছে এবং এটি রেজিস্টার R2 এর মধ্যে লোড করা হয়। তারপর R2 কে 1 দ্বারা হ্রাস করা হয়, তাই R2 এখন 4। এবং 4 এর মান পুনরায় counter এর মধ্যে সংরক্ষিত হয়। তাই এখন, মেমরির counter টি 4 হল। এখন একটি context switch হল এবং মনে করুন যখন কোন context switch হবে তখন প্রোগ্রাম 0 যেখানে থেমেছিল সেখান থেকে শুরু হবে। মূলত, context টি

kernel এ সংরক্ষণ করা হয়েছিল যেটি প্রোগ্রাম 0 তে ফিরিয়ে আনা হয়। অতএব, আমরা দেখি যে এই নির্দিষ্ট বিন্দুতে R1 এর মান ছিল 5 এবং context switch এর পরে R1 এর মান 5 পাওয়া গেল। এবার R1 এর মান বাড়িয়ে হল 6 এবং 6 মানটি counter এ সংরক্ষণ করা হয়। অতএব, এই execution এর শেষে counter এর মান হল 6। এখন আসুন আমরা তৃতীয় কেসটি দেখি, যখন আমাদের counter এর মান 4। এটি দ্বিতীয় ক্ষেত্রের বিপরীত যেখানে প্রোগ্রাম 1 প্রথম execute করা হয়। সুতরাং মূলত counter এর মান 5 যা R2 তে লোড হয়, সুতরাং R2 এ রয়েছে 5। এখন একটি context switch হল যার ফলে প্রোগ্রাম 0 চালানো হয় এবং counter এর মান R2 তে লোড হয় এবং R2 এর মান 1 দ্বারা বাড়ানো হয়, R2 এর মান হয় 6 এবং counter এর মান হল 6। এবার একটি context switch হল আর প্রোগ্রাম 1 যেখানে বন্ধ হয়েছিল সেখান থেকে চালানো হল। R2 এর মান ছিল 5 এখন, R2 1 দ্বারা হ্রাস করা হয়, ফলে এটি 4 হয়। এবং 4 এর মানটি counter এর মেমরি অবস্থানে সঞ্চিত হয়। সুতরাং, এই execution এর শেষে counter এর মান 4 হয়। সুতরাং, এই সমস্যা একটি উদাহরণ যেখানে shared memory আছে। যদিও এক জায়গায় কাউন্টার বৃদ্ধির এবং অন্য জায়গায় কাউন্টার হ্রাসের একটি সহজ অপারেশন ছিল আমরা দেখেছি কীভাবে নির্দেশাবলী কার্যকর করা যায় এবং কীভাবে context switch ঘটেছে তার উপর নির্ভর করে ফলাফলটি ভিন্ন হতে পারে,। তাই আমরা এই পরিস্থিতি আরও আনুষ্ঠানিকভাবে সংজ্ঞায়িত করব যা race condition হিসাবে পরিচিত।

(স্লাইড সময় 07:43 দেখুন)

Race condition এমন একটি অবস্থা যেখানে বিভিন্ন প্রসেস একই তথ্য access এবং manipulate করে। প্রসেসের যে অংশ যা সাধারণ বা shared data অ্যাক্সেস করে সেটি critical section হিসাবে পরিচিত। Race condition এর ফলাফল data access এর ক্রমের উপর নির্ভর করে। আমরা পূর্ববর্তী উদাহরণে দেখেছি, কোন প্রোগ্রাম প্রথমে execute করে এবং context switch কীভাবে ঘটবে তার উপর ভিত্তি করে ফলাফলটি ভিন্ন হবে। Synchronization দ্বারা race condition প্রতিরোধ করা যেতে পারে। মূলত, synchronization এর মাধ্যমে আমরা নিশ্চিত করতে পারি যে কোন এক সময়ে শুধুমাত্র একটি প্রসেস critical section এর data manipulate করবে। তাই আমাদের উদাহরণে এটি করা প্রয়োজন, যে instruction এর যে অংশে shared data access বা manipulate করা হয় তাকে আমরা critical section হিসেবে চিহ্নিত করব। তারপর আমরা কিছু অতিরিক্ত কৌশলে নিশ্চিত করব যে একাধিক প্রসেস একই সময়ে একটি critical section এ execute না করতে পারে। আমরা পরবর্তী ভিডিওগুলিতে এটিকে আরো বিস্তারিতভাবে দেখব।

(স্লাইড সময় 09:09 দেখুন)

শুধুমাত্র single core সিস্টেমেই race condition হয় তা নয় এটি multicore সিস্টেমেও হয়। এটি খুবই স্পষ্ট যে একটি মাল্টিকোর সিস্টেমের প্রতিটি প্রসেসর একসাথে চালানো হয়, তাই একই সময়ে উভয় প্রোগ্রামই shared variable কে access করতে পারে। অতএব, একক কোর সিস্টেমের তুলনায় মাল্টি কোর সিস্টেমে race condition আরো প্রকট।

(স্লাইড সময় 09:42 দেখুন)

এখন, আসুন দেখি কীভাবে critical section সমস্যার সমাধান পাওয়া যেতে পারে। Critical section সমস্যার কোন সমাধানের জন্য নিম্নলিখিত শর্তাবলী পূরণ করা প্রয়োজন। এক নম্বর হল mutual exclusion, দ্বিতীয়ত progress এবং তৃতীয়ত No starvation বা bounded wait। Mutual exclusion অর্থ কোন critical section এর সমাধানটিকে নিশ্চিত করতে হবে যে নির্দিষ্ট সময়ে একাধিক প্রসেস critical section এ নেই। Progress এর অর্থ এটি নিশ্চিত করতে হবে যে যখন কোনও প্রসেস critical section এ নেই তখন কোন প্রসেস critical section এ প্রবেশ করতে চাইলে কোনও বিলম্ব ছাড়াই অনুমতি প্রদান করা উচিত। No starvation বা bounded wait এর অর্থ একটি প্রসেস যখন অপেক্ষা করছে অন্য একটি প্রসেস কতবার critical section এ প্রবেশ করছে তার একটি উর্ধসীমা থাকবে। অর্থাৎ critical section এ প্রবেশের জন্য একটি প্রসেসকে যেন অসীম সময় অপেক্ষা করতে না হয়।

(স্লাইড সময় 10:44 দেখুন)

সমস্ত critical section সমস্যাগুলি সমাধানের জন্য locking এবং unlocking হিসাবে পরিচিত দুইটি কৌশল ব্যবহার করে। মূলত, সমাধানের জন্য এই রকম কিছু থাকবে যা একটি shared variable। তাই আমরা L বলে একটি lock কে সংজ্ঞায়িত করব। তাই critical section এ প্রবেশ করার আগে প্রোগ্রাম লকটিকে (lock(L)) চালু করবে এবং critical section থেকে প্রস্থান করার সময় unlock(L) করবে। অনুরূপে critical section ব্যবহার করে এমন প্রত্যেকটি প্রোগ্রাম critical section এ প্রবেশ এবং প্রস্থান করার পূর্বে যথাক্রমে lock এবং unlock করবে। এখন lock কি করে, এটি lock L কে exclusively অর্জন করে। সুতরাং, function lock L সম্পূর্ণ কার্যকর হওয়ার পর এটি নিশ্চিত হয়ে যায় যে একটি প্রসেস, এই ক্ষেত্রে শুধুমাত্র এই প্রসেসটি critical section এ প্রবেশ করে এবং সেখানে থাকে। যখন unlock চালু করা হয় তখন lock এর exclusive access ছেড়ে দেওয়া হয়, এর ফলে অন্য প্রসেস critical section এ প্রবেশ করতে পারে। এখন লকিং এবং আনলকিং এই উপায়ে পরিকল্পিত হওয়া উচিত যে critical section এর সমাধানগুলির তিনটি প্রয়োজনীয়তা যথা mutual exclusion, progress এবং bounded wait মান্য হয়। আমরা ইতিমধ্যে mutual exclusion দেখেছি এবং এবার progress দেখা যাক। মনে করি যে প্রোগ্রামটি 0 টি critical section এ নেই এবং প্রোগ্রাম 1 এখানে এসেছে এবং এটি লককে অনুরোধ করেছে তাই progress মানে এই যে, যেহেতু কোনও অন্য প্রোগ্রাম critical section এ নেই তাই এটিকে

lock অবিলম্বে দেওয়া উচিত। সুতরাং, প্রোগ্রাম 1 কে lock এর exclusive access লক একচেটিয়া অ্যাক্সেস পেতে হবে। Bounded wait বৃত্তে একটি অবস্থা দেখা যাক। মনে করি প্রোগ্রাম 0 critical section এ উপস্থিত অবস্থায় প্রোগ্রাম 1 lock কে অনুরোধ করেছে। প্রোগ্রাম 1 এর জন্য অপেক্ষা করার সীমা আছে যতক্ষণ না এটি critical section এ access পায়, যে সমাধানটি নিশ্চিত করে যে প্রোগ্রাম 0 L কে খুলে দেবে এবং শুধুমাত্র তারপর প্রোগ্রাম 1 lock এ প্রবেশ করবে। সুতরাং, যদি প্রোগ্রাম 1 লককে অনুরোধ করে এবং প্রোগ্রাম 0 ইতিমধ্যে critical section এ রয়েছে তবে প্রোগ্রাম 1 এর অপেক্ষা করার সময়ের একটি ঊর্ধ্বসীমা থাকবে।

(স্লাইড সময় 13:26 দেখুন)

একটি প্রোগ্রামে লক এবং আনলক নির্মাণের ব্যবহার নিশ্চিত করবে যে critical section টি অবিভাজ্য। সুতরাং কখন এই লকিং প্রক্রিয়া ব্যবহার করতে হবে? মূলত একটি একক instruction নিজেই অবিভাজ্য। যেমন ebx এর সাথে eax যোগ করা একটি অবিভাজ্য instruction এবং এটি স্পষ্ট যে এর জন্য লকিং প্রয়োজন হয় না। যাইহোক, একাধিক instructions যেখানে instructions এর একটি ক্রম আছে এবং যদি তাদের অবিভাজ্য করতে হয় তাহলে explicit locking এবং unlocking প্রয়োজন, তাই অপারেটিং সিস্টেমের প্রতিটি কোড পরীক্ষা করতে হবে যে তাদের অবিভাজ্য করা প্রয়োজন কি না। Interrupt handler এর নির্দেশগুলি অবিভাজ্য হবে কি না তা পরীক্ষা করতে হবে। এই বিশেষ ভিডিওতে, আমরা instructions লকিং এবং আনলক করার জন্য প্রয়োজনীয়তা দেখেছি। পরবর্তী ভিডিওতে, আমরা দেখতে যাচ্ছি কিভাবে লকিং এবং আনলকিং পদ্ধতি সিস্টেমে প্রয়োগ করা হয়। ধন্যবাদ।